

Predicting Ligue 1 Match Outcomes with Player-Level Statistics

Data Collection & Feature Organisation

Sienna O'Shea, Samuel Majbruch, Ioana Olaru & Ava Sudit

Ligue 1 · Seasons 2018–2024 · 1,653 matches · 73 features

Outline

- 1 Research Hypothesis
- 2 Data Sources
- 3 Dataset Overview
- 4 Feature Engineering
- 5 Feature Selection
- 6 Modeling
- 7 Key Findings

Central Hypothesis

Research Question

Do **player-level** rolling statistics add predictive power for match outcomes beyond team-level aggregates and bookmaker odds?

Standard approach

Team-level rolling form · squad market value · bookmaker implied probabilities

This project adds

Per-player rolling stats *tracked across transfers*, aggregated over the starting XI weighted by minutes played

Key insight

Tracking individual player form (not team form) captures information like mid-season transfers, rotation, and injury recovery that team aggregates smooth over.

Data Sources — Three Pipelines

Understat

What: xG timelines, match results, pre-match win/draw/loss probabilities

Seasons: 2018–2023

Rows: 2,105 matches

Script:
`collect_understat.py`



Transfermarkt

What: Squad market values, player counts, squad age

Seasons: 2018–2023

Rows: 174 team-seasons

Script:
`collect_transfermarkt.py`



FBref

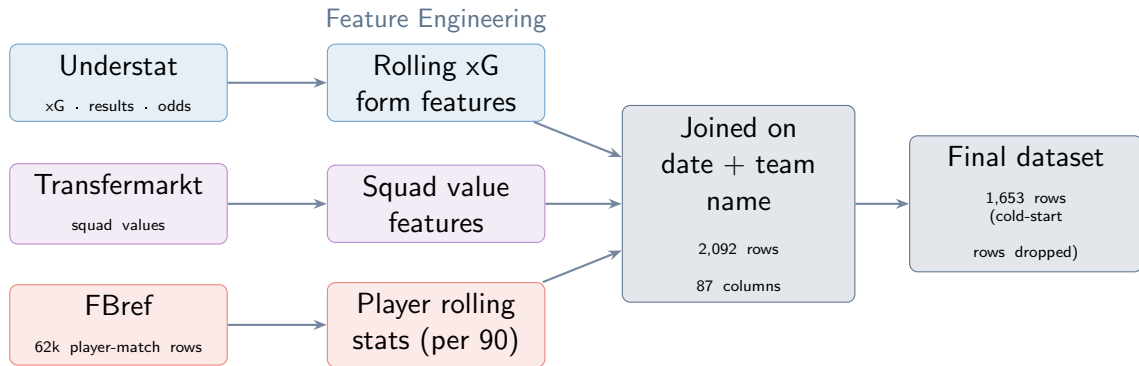
What: Per-player per-match: goals, assists, shots, SoT, tackles, interceptions, minutes, position

Rows: 62,683 player-match rows

Script: `collect_fbref.py`

FBref data was scraped via soccerdata using an undetected headless Chrome driver; the 2021/22 season required a visible browser pass to resolve a CAPTCHA block.

Data Pipeline



Dataset at a Glance

Coverage

Matches	1,653
Seasons	2018–2024
Teams	29 (incl. promoted)
Features	73
Target	H / D / A

Feature sources

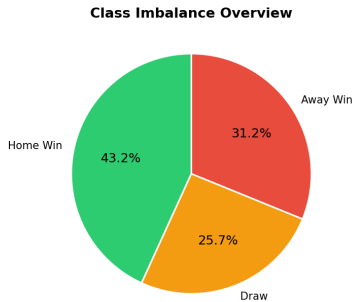
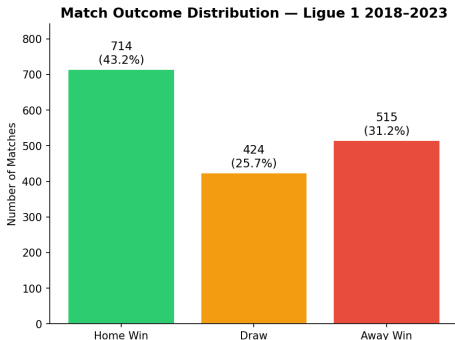
Understat / Transfermarkt	32
FBref team-level	17
Player-level	24

Season breakdown

Season	Matches	Note
2018/19	370	
2019/20	278	COVID — curtailed
2020/21	379	
2021/22	379	FBref initially blocked
2022/23	380	
2023/24	306	Partial (to May 2024)

Plot 1 — Target Variable Distribution

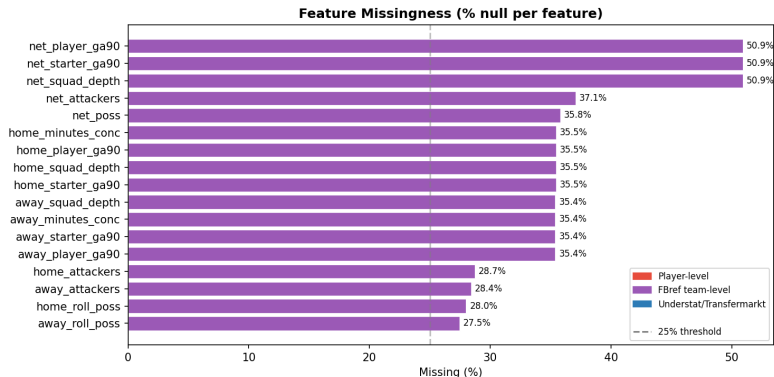
Total matches: 1653 | Seasons 2018-2023 | 29 teams



Class imbalance

Draws are underrepresented at $\approx 26\%$. Models must use class weights or calibrated probabilities — accuracy alone is a misleading metric.

Plot 3 — Missing Data: Sources & Resolution



FBref team features (~28–36% null):
possession and formation data for 2021/22 was not scraped — FBref blocked that season.
Handled via imputation.

Player rolling stats: cold-start nulls (first ≈ 3 matchdays/season). Fixed by dropping those 439 rows, leaving 1,653 clean matches.

Feature Engineering — Three Stages

Stage 1 — Understat + Transfermarkt

Rolling 5-match xG, goals, wins, points per team. Squad market value, average player value, squad age. Bookmaker implied win/draw/loss probabilities.

Stage 2 — FBref team-level

Rolling possession, formation (attackers count), player season G+A/90 aggregated to team level.

Stage 3 — Player-level (novel contribution)

Per-player rolling shots, SoT, defensive actions, goals, assists per 90 tracked *across team changes*. Aggregated over the starting XI weighted by minutes played. Net differentials (home — away) for each stat.

All rolling windows use `shift(1)` before rolling mean — the current match is *never* included in its own features.

Player-Level Aggregation Logic

Per player, prior 5 appearances:

$$\text{roll}_{p,t} = \frac{1}{5} \sum_{k=1}^5 \text{stat}_{p,t-k}$$

Tracked *individually* — a transfer does not reset form.

Starting XI aggregate (weighted mean):

$$\hat{s}_{\text{team}} = \frac{\sum_{p \in \text{XI}} \min_p \cdot \text{roll}_p}{\sum_{p \in \text{XI}} \min_p}$$

Player features produced

*_sh_p90	shots / 90
*_sot_p90	shots on target / 90
*_def_p90	TkIW + Int / 90
*_gls_p90	goals / 90
*_ast_p90	assists / 90
*_starter_mins	regularity proxy
*_pct_fw	fraction forwards
*_depth_sh	shot depth gap

Each exists as home_pl_*, away_pl_*, net_pl_*

Feature Selection — Four-Method Protocol

Each of the 73 features was scored by **four independent methods**, percentile-normalised (0–1), and averaged into a **composite score**.

1. Mutual Information (Filter)

How much does knowing this feature reduce uncertainty about the result? Non-parametric; catches non-linear relationships.

2. RFECV — Wrapper

A Random Forest eliminates weak features iteratively under temporal CV. Only features it *keeps* survive. Optimal: **26 features**.

Temporal CV (TimeSeriesSplit, 5 folds) throughout — data never shuffled across time.

3. LASSO (Embedded)

Penalised logistic regression zeroes redundant features. 17 features zeroed; **0 player features** zeroed.

4. RF Importance (Embedded)

Mean decrease in impurity across 300 trees. Stable across seeds; corroborates RFECV ranking.

Plot 4 — Feature Distributions by Match Outcome (Top 20)



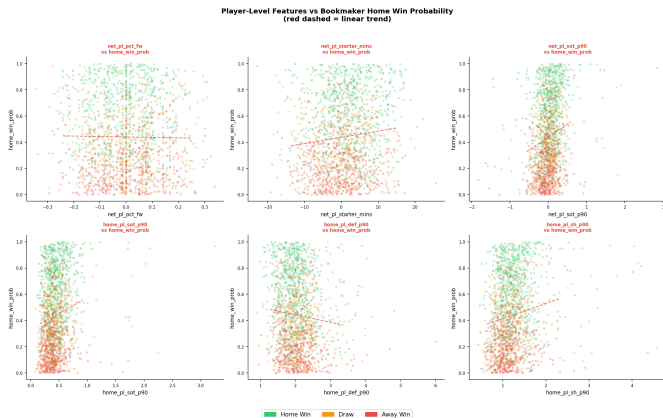
How to read: each panel shows the distribution of one feature split by match outcome (H/D/A).

Less overlap between the three coloured distributions $\Rightarrow$ more discriminative feature.

Bookmaker probability features show the clearest separation.

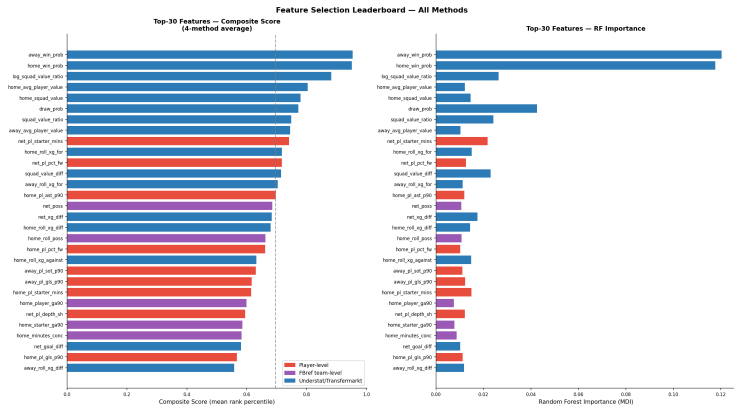
Player-level features add a small but consistent nudge on top of the bookmaker signal.

Plot 8 — Bookmaker Odds vs Player-Level Features



The bookmaker's implied probability and the player rolling stats **mostly disagree** — wide scatter, no tight trend. This is a good sign: player features are contributing *new information* that the bookmaker does not fully price in.

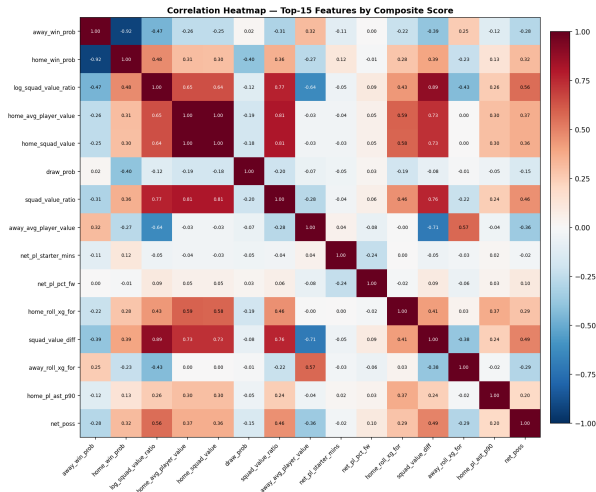
Plot 12 — Feature Leaderboard: All 73 Features Ranked



■ Understat/Transfermarkt ■ FBref team-level ■ Player-level

Two player features enter the top 10: `net_pl_starter_mins` (rank 9) and `net_pl_pct_fw` (rank 11). 10 player features survive RFECV wrapper selection.

Plot 14 — Correlation Heatmap: Top 15 Features



Red zones = high correlation $\Rightarrow$ redundant (e.g. `home_win_prob` and `away_win_prob` are two sides of the same bet).

Player features (`net_pl_pct_fw`, `net_pl_starter_mins`) show *near-white* rows and columns — barely correlated with anything else.

This confirms they bring **genuinely new information** rather than repeating the bookmaker signal.

Modeling — Protocol

Three feature regimes compared on the **same** 593-match common subset (NaN-free across all three CSVs), with a strict temporal holdout: train on seasons 2018–2022 (425 matches), test on season 2023/24 (168 matches).

Models

Baseline: Understat pre-match win probabilities.

Linear: multinomial logistic regression (scaled, $C = 1$).

Trees: Random Forest, XGBoost, and a 50/50 **ensemble** of the two.

Tuning & calibration

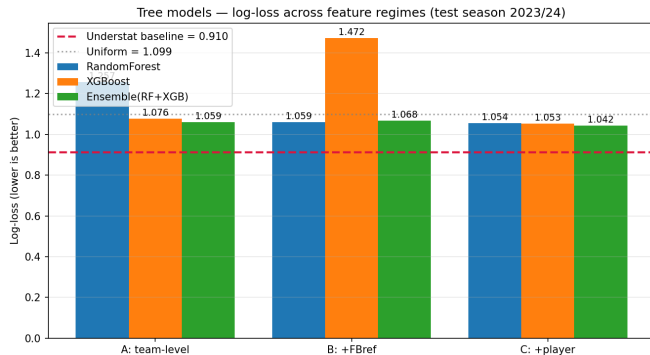
Search: RandomizedSearchCV (100 configs) over TimeSeriesSplit(5), scored by log-loss.

Imbalance: class weights (RF) / sample weights (XGB) to handle rare draws.

Calibration: isotonic via CalibratedClassifierCV on a temporal inner split.

Metrics: log-loss (primary, lower is better), Brier, accuracy. Uniform reference = 1.099, Understat baseline = 0.910.

Modeling — Results



Test log-loss (season 2023/24)

	A	B	C
RF	1.26	1.06	1.05
XGB	1.08	1.47	1.05
Ensemble	1.06	1.07	1.04
LR (ref)	1.06	1.14	1.22

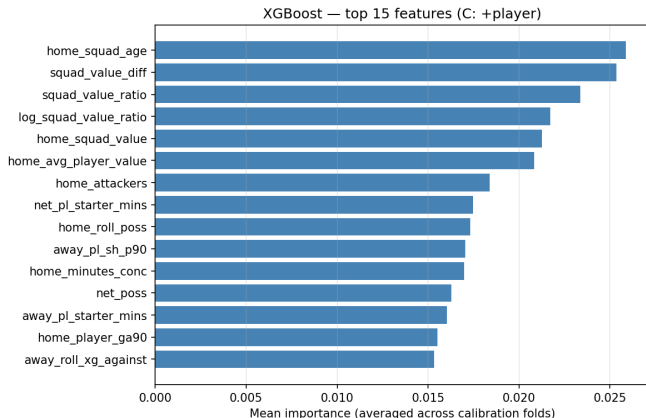
A=team, **B**=+FBref, **C**=+player

Accuracy in context (3-class):

Random guess	33%
Always home	43%
Best model	50%
Bookmaker	56%

No model clears the bookmaker baseline (0.91 log-loss). Bookmakers have real-time info (injuries, team news) that rolling historical stats cannot capture.

Modeling — Feature Importance (XGBoost, full regime)



How to read: mean importance over the calibration folds for the best XGBoost on the full player regime.

Team-level form (rolling xG, market value) sits at the top, as expected.

Player-level features (`starter_mins`, `pct_fw`, `depth_gaps`) appear repeatedly in the top ranks — they are not just noise the model tolerates, they are signal the model actively uses.

Consistent with the feature-selection leaderboard (`net_pl_starter_mins` rank 9, `net_pl_pct_fw` rank 11).

Modeling — What changed vs Logistic Regression

1. Tree models flip the feature-regime story

Logistic regression *got worse* as features were added ($1.06 \rightarrow 1.14 \rightarrow 1.22$): multicollinearity and curse of dimensionality. Tree models *improve* with more features (RF: $1.26 \rightarrow 1.06 \rightarrow 1.05$) — they handle correlated inputs natively.

2. Player features add real signal

The full player regime (C) is the best for every tree model. Confirms the project hypothesis: player-level rolling stats add predictive power once the model can use them.

3. Ensembling extracts a final gain

RF and XGB make different mistakes; averaging their calibrated probabilities beats either alone (1.04 vs $1.05/1.05$ in regime C).

Key Findings

The data pipeline

62,683 FBref player-match rows across 6 seasons, joined to Understat xG and Transfermarkt squad values → 73-feature, 1,653-match dataset with **zero null values** in all player rolling features.

Answer to the hypothesis — Yes

Selection: `net_pl_starter_mins` (rank 9) and `net_pl_pct_fw` (rank 11) survive all four methods; 10 player features kept by RFECV.

Independence: player features are near-zero correlated with the bookmaker signal — new information, not a repeat.

Predictive lift: adding player features *improves* every tree model's log-loss on a held-out 2023/24 season, reversing the logistic-regression result.

Best model

Ensemble of RF + XGBoost on the full player regime: **log-loss 1.04**, accuracy 50.6% — closes ~**33%** of the uniform-to-Understat gap (1.10 → 0.91) using only public data.